

DSA STUDY BLUEPRINT SERIES

Stock Trading & Binary Exponentiation

Greedy Approach and Logarithmic Computations

Section 06 • C++ Core Data Structures & Algorithms

Best Time to Buy and Sell Stock

Maximizing profit with single transactions (LeetCode 121)

The Greedy Intuition

- We want to buy at the lowest possible price and sell at the highest.
- We must buy **before** we sell (no backtracking in time).
- Track the minimum price seen so far.
- Calculate potential profit at each step and update maximum profit.

Algorithmic Steps $O(N)$

- Initialize `min_price = prices[0]`, `max_profit = 0`.
- Traverse array from index 1 to $N-1$.
- Update `min_price = min(min_price, prices[i])`.
- Update `max_profit = max(max_profit, prices[i] - min_price)`.

Power(x, n) via Binary Exponentiation

Computing x raised to power n in $O(\log N)$ time

Linear vs Logarithmic Math

- **Linear:** Multiply x , n times. Takes $O(N)$ operations.
- **Binary Exponentiation:** Divide power in half each step:
 - If n is even: $x^n = (x^2)^{n/2}$
 - If n is odd: $x^n = x * x^{n-1}$

Integer Overflow Concerns

- Power parameter `n` can be negative. We convert it to positive.
- If `n = INT_MIN`, converting it to positive exceeds `INT_MAX`.
- Solution: Cast `n` to a 64-bit integer (`long long`) before manipulation.

C++ Implementations

Stock Profit and Fast Power code snippets

```
// Best Time to Buy and Sell Stock - O(N) Time, O(1) Space
int maxProfit(vector& prices) {
    int min_price = prices[0];
    int max_profit = 0;
    for (int price : prices) {
        min_price = min(min_price, price);
        max_profit = max(max_profit, price - min_price);
    }
    return max_profit;
}
```

```
// Power(x, n) using Binary Exponentiation - O(log N) Time
double myPow(double x, int n) {
    long long nn = n;
    if (nn < 0) nn = -nn, x = 1.0 / x;
    double ans = 1.0;
    while (nn > 0) {
        if (nn % 2 == 1) ans *= x, nn--;
        else x *= x, nn /= 2;
    }
    return ans;
}
```